

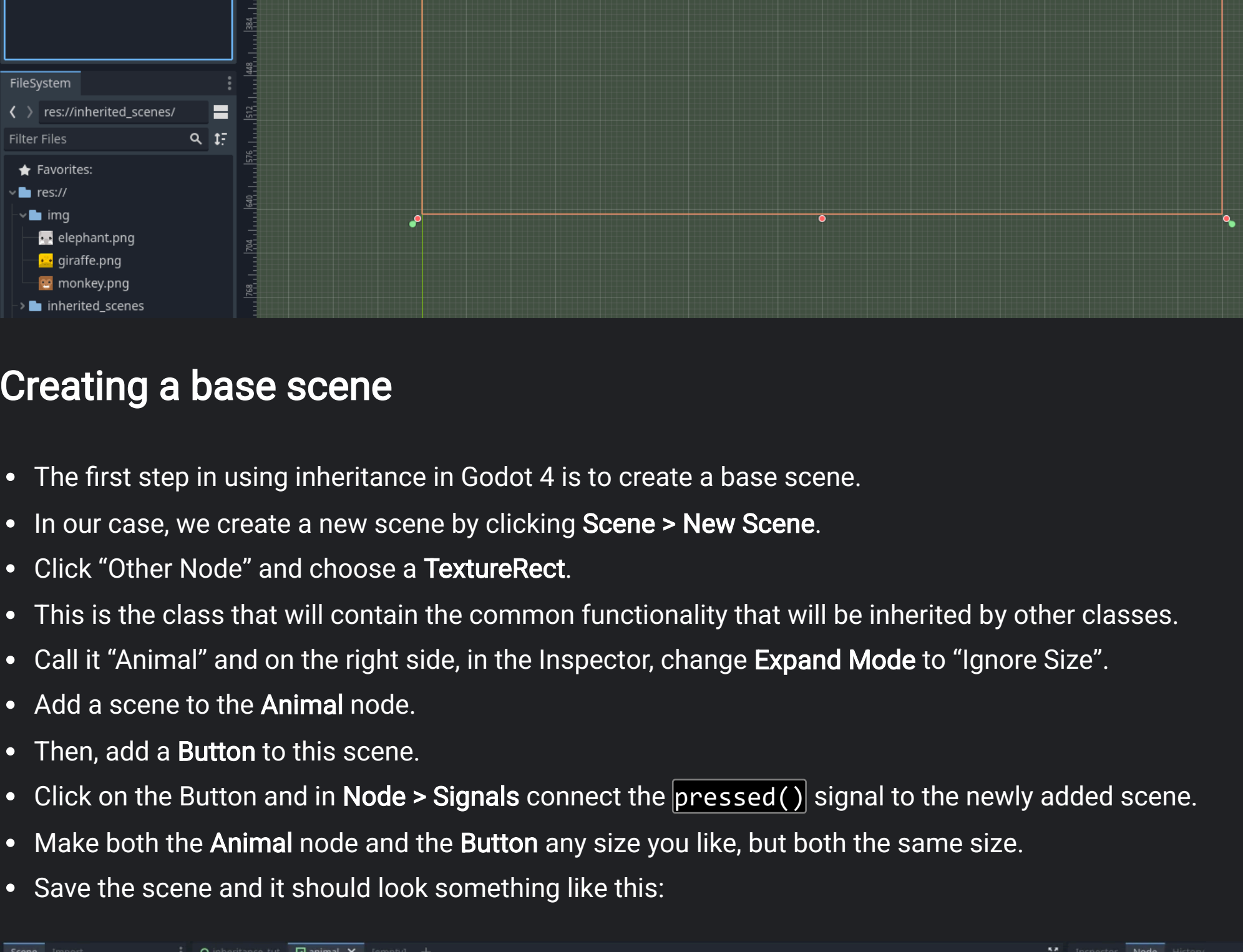
Inheritance in Godot 4 – Tutorial

Inheritance is an important concept in game development, as it allows you to reuse code and create a hierarchy of classes with similar functionality. Godot 4 offers powerful inheritance capabilities that can help you create efficient and flexible code structures. In this tutorial, we will explore how to use inheritance in Godot 4 and how it can make your game development process easier and more streamlined.

Dear reader, please consider supporting my game 

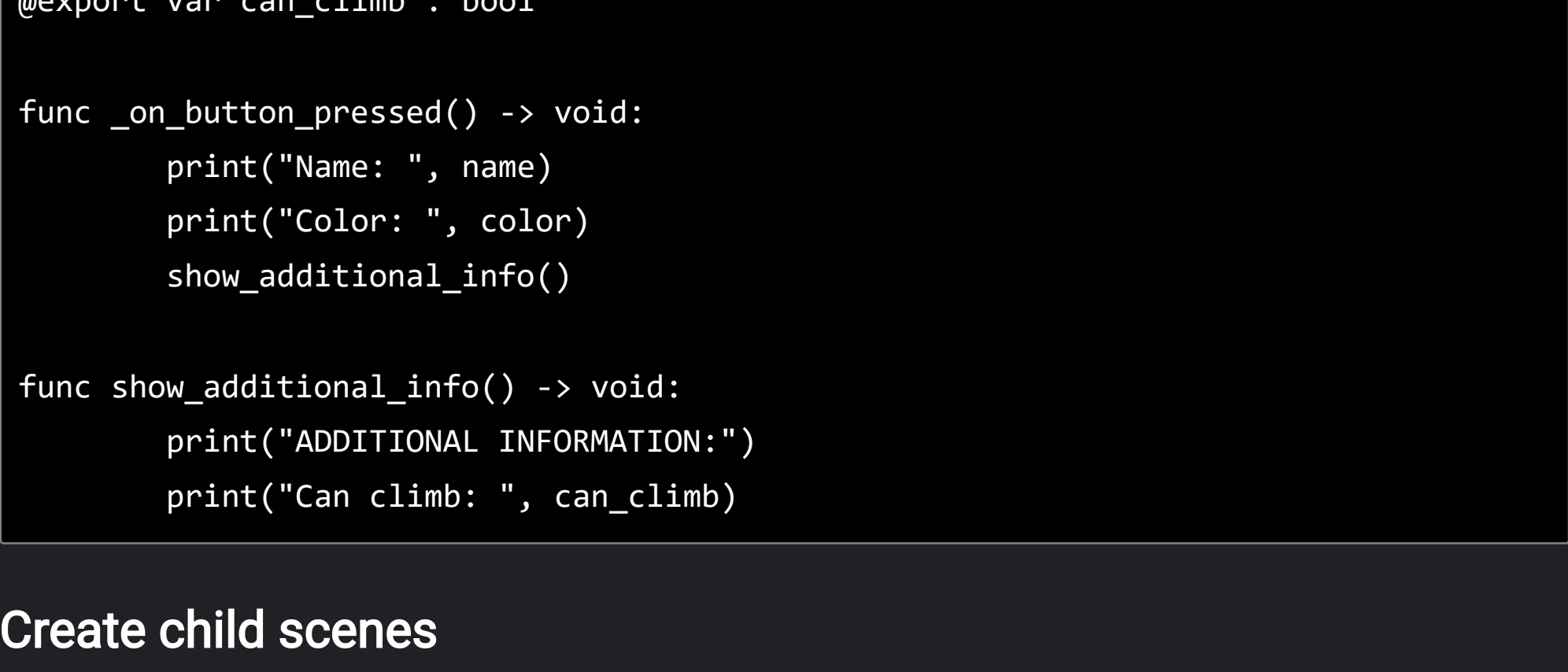
Create a scene for the inheritance tutorial

- Create a new project, go to **Scene > New Scene** and create a **Control** Node.
- Call it “InheritanceTut” or however you like.
- For the purpose of this tutorial, I downloaded three animal heads from kenney.nl.
- Add them (or any other three images) to your project into a folder called “img”.
- Create another folder inside your project and call it “inherited_scenes”.
- It should look like the following:



Creating a base scene

- The first step in using inheritance in Godot 4 is to create a base scene.
- In our case, we create a new scene by clicking **Scene > New Scene**.
- Click “Other Node” and choose a **TextureRect**.
- This is the class that will contain the common functionality that will be inherited by other classes.
- Call it “Animal” and on the right side, in the Inspector, change **Expand Mode** to “Ignore Size”.
- Add a scene to the **Animal** node.
- Then, add a **Button** to this scene.
- Click on the Button and in **Node > Signals** connect the `pressed()` signal to the newly added scene.
- Make both the Animal node and the Button any size you like, but both the same size.
- Save the scene and it should look something like this:



- Add the following to the script:

```
extends TextureRect
class_name Animal # We give this script a unique class name

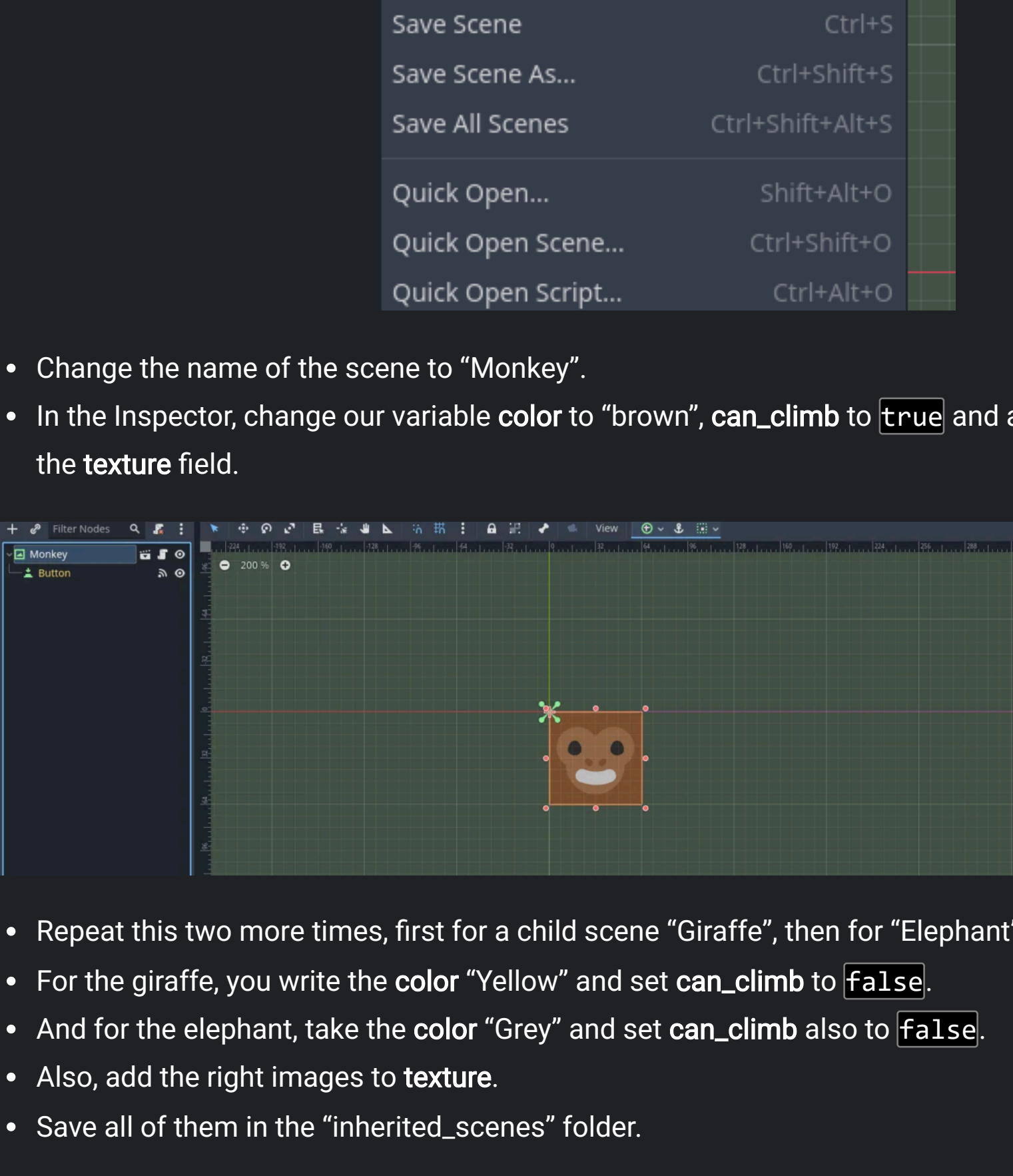
@export var color : String
@export var can_climb : bool

func _on_button_pressed() -> void:
    print("Name: ", name)
    print("Color: ", color)
    show_additional_info()

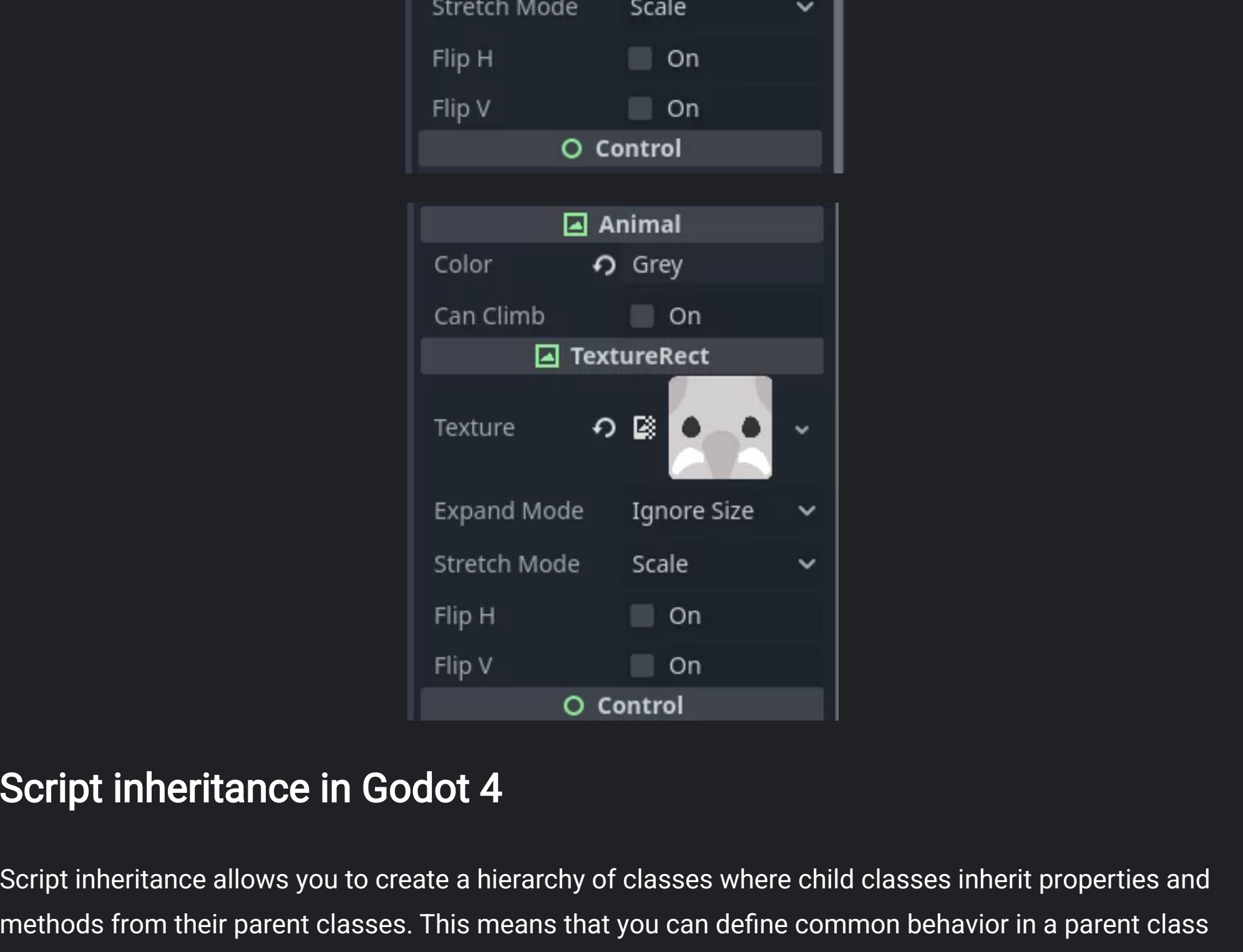
func show_additional_info() -> void:
    print("ADDITIONAL INFORMATION:")
    print("Can climb: ", can_climb)
```

Create child scenes

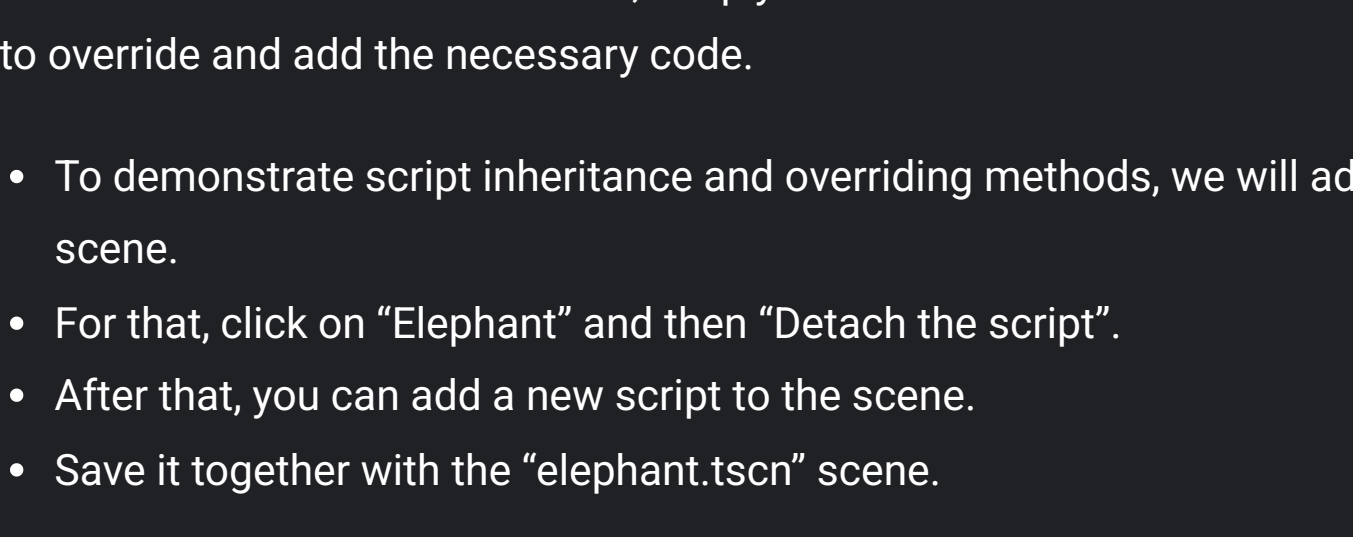
- Once you have created your base scene, you can create child scenes that inherit from it.
- To create a child scene, simply right-click on “animal.tscn” in the **FileSystem** and click on “New Inherited Scene”.
- Or you can go to **Scene > New Inherited Scene** and then choose the “animal.tscn” file.
- Either way, a new scene will be opened.



- Change the name of the scene to “Monkey”.
- In the Inspector, change our variable `color` to “brown”, `can_climb` to `true` and add the right image to the `texture` field.



- Repeat this two more times, first for a child scene “Giraffe”, then for “Elephant”.
- For the giraffe, you write the `color` “Yellow” and set `can_climb` to `false`.
- And for the elephant, take the `color` “Grey” and set `can_climb` also to `false`.
- Also, add the right images to `texture`.
- Save all of them in the “inherited_scenes” folder.



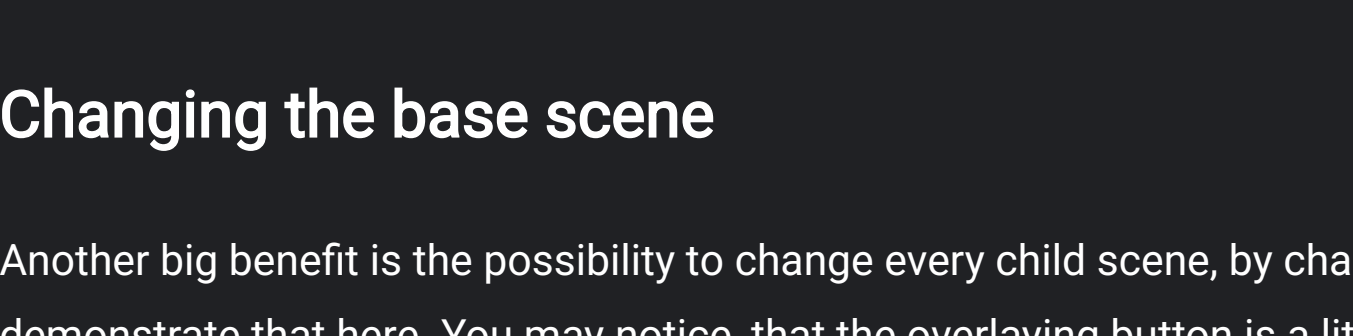
Script inheritance in Godot 4

Script inheritance allows you to create a hierarchy of classes where child classes inherit properties and methods from their parent classes. This means that you can define common behavior in a parent class and then extend that behavior in child classes, reducing code duplication and making it easier to maintain and update your code. You can achieve this by creating a hierarchy of classes using the `extends` keyword. And to create a new class, use the `class_name` keyword, as we have done with “Animal”.

Overriding methods

One of the key benefits of inheritance is the ability to override methods in child classes. This allows you to modify or extend the behavior of the base class without having to modify the base class itself. To override a method in a child class, simply create a method with the same name as the method you want to override and add the necessary code.

- To demonstrate script inheritance and overriding methods, we will add a new script to the elephant scene.
- For that, click on “Elephant” and then “Detach the script”.
- After that, you can add a new script to the scene.
- Save it together with the “elephant.tscn” scene.



- Then, add the following to this script:

```
extends Animal # make this scene inherit from our Animal class

@export var has_trunk : bool

func show_additional_info() -> void:
    super.show_additional_info() # call the parent function
    print("has trunk: ", has_trunk)
```

- Make sure to give the elephant the right information and image in the **Inspector** again.

Calling super methods

In some cases, you may want to call the method of the base class in your child class. This is where the `super` keyword comes in. By calling the `super` method, you can execute the method of the base class before executing the code in the child class. For example, we have overridden the `show_additional_info()` method but with the `super.show_additional_info()` method we still can execute the original function.

Polymorphism

Another benefit of inheritance is polymorphism, which allows you to use child classes in place of the base class. This means that you can pass instances of child classes to methods that expect instances of the base class, and they will still work as expected. In our example, if you have a method that expects an instance of the `TextureRect` class, you can pass an instance of the `Animal` class instead and it will still work.

Changing the base scene

Another big benefit is the possibility to change every child scene, by changing the base scene. Let me demonstrate that here. You may notice, that the overlaying button is a little bit dark.

- So let's change that by opening the base scene “animal.tscn”.
- Click on the **Button** and under **Visibility > Modulate**, change the alpha channel.

- Save the scene.
- Now you can check the child scenes, and you will find, that they all have the new opacity.

But keep in mind: If you have changed the opacity in the child scene prior to the change in the parent scene, the change will not affect this child. You can use the round arrow button to reset it to the base scene, though.

Wrapping up the inheritance tutorial

- Now add all three animal scenes to the main scene (**InheritanceTut**) and place them as you wish.
- Save everything and play the scene.
- It should work like the following:

And we are done. Inheritance is a powerful tool in Godot 4 that can help you create flexible and efficient game and code structures. By creating base scenes and scripts as well as child scenes and scripts, overriding methods, and using polymorphism, you can create a hierarchy that share common functionality while still allowing for customization and flexibility. So take the time to explore inheritance in Godot 4 and see how it can improve your game development process.